

Lab 3: Prefetcher

Peiyu Liu

December 20, 2024

1 Introduction

In this project, we explored the design of prefetchers in computer systems. Using the gem5 simulator, we evaluated a variety of prefetchers across multiple performance metrics and reproduce a recently published paper that introduces a high-performance prefetcher, Hyperion. Experimental results demonstrated that Hyperion outperforms the stride prefetcher across most measured metrics and performs comparably to the AMPM prefetcher.

2 Experiment Setup

We first built a cache hierarchy, which includes L1 and L2 caches, and evaluated the impact of cache on performance. Next, we attached prefetchers to the L1D cache. To ensure a fair comparison of the performance across different prefetchers, we ran two memory-intensive benchmark programs—gemm and shellsort—using identical configurations for the CPU (O3CPU), L1D and L2 caches, and memory (DDR4). The only variable in the setup was the type of prefetcher used in the L1D cache. Prefetchers used in this project is provided in Table 1.

Prefetcher	Strategy	Built-in / Customized
Stride	Delta prefetching	Built-in
AMPM	Access map pattern matching	Built-in
Hyperion	Timely and fine-grained delta prefetching	Customized

Table 1: Prefetcher Strategies

As shown in Table 1, we developed a prefetcher, Hyperion, using the gem5 simulator. Hyperion replicates the approach described in the paper by Y. Cui *et al.*. In Section 4, the paper will be summarized in detail from backgrounds and design.

3 Results

3.1 Adding a Cache Hierarchy

First, we constructed a cache hierarchy and compared the IPC of CPUs with and without the cache hierarchy using the gemm benchmark. Figure 1 shows a screenshot confirming the successful integration of the cache hierarchy, and the results are presented in Table 2. As expected, the CPU with a cache hierarchy demonstrates significantly better performance.

gemm	w/o cache	w/ cache
IPC	0.150984	1.182400

Table 2: IPC of CPUs with and without Cache

```
[system.cpu]
type=Base03CPU
children=branchPred dcache decoder fuPool icache interrupts isa l2cache mmu power_state toL2Bus tracer workload
```

Figure 1: Screenshot of the Cache Configuration

3.2 Attaching a Prefetcher to the L1D Cache

Next, we attached different types of prefetchers to the L1D cache and evaluated their performance. Similarly, we compared the IPC of CPUs with and without a prefetcher using the gemm and shellsort benchmarks. Additionally, we compared two built-in prefetchers in gem5: the Stride Prefetcher and the AMPM Prefetcher. The results are shown in Table 3 and Table 4.

gemm	w/o Prefetcher	Stride	AMPM
IPC	1.182400	1.183529	3.125727
Accuracy	-	0.000255	0.073765
Coverage	-	0.000002	0.983178
Late Count	-	9965	31154152

Table 3: Comparison of Metrics for Different Prefetchers on the gemm Benchmark

shellsort	w/o Prefetcher	Stride	AMPM
IPC	0.357822	0.392248	0.687109
Accuracy	-	0.300289	0.237503
Coverage	-	0.074525	0.708856
Late Count	-	3000534	31094045

Table 4: Comparison of Metrics for Different Prefetchers on the shellsort Benchmark

As expected, CPUs with prefetchers show better performance compared to those without a prefetcher, with the AMPM Prefetcher generally outperforming the Stride Prefetcher (Although the Stride Prefetcher achieves higher accuracy than the AMPM Prefetcher on the shellsort benchmark, its lower coverage leads to a smaller IPC compared to the AMPM Prefetcher). We used three metrics to evaluate the performance of each prefetcher: *Accuracy*, which measures the fraction of prefetches where the data is accessed before eviction; *Coverage*, which indicates the prefetcher’s ability to reduce cache misses that would occur without prefetching; and *Late Count*, which tracks the number of prefetches issued later than the actual data access. Together, these metrics reflect the prefetcher’s ability to detect and exploit memory access patterns in the programs.

3.3 Performance of Hyperion

Hyperion is a timely delta prefetcher that leverages fine-grained information and context to precisely recognize memory access patterns. The implementation of Hyperion, as described in the paper, will be discussed in detail in Section 4. Our customized implementation is largely the same as the one in the paper, except for certain features that were not required for our implementation. Most of the fine-tuning was done based on the documentation; however, to thoroughly assess Hyperion’s performance, we did not increase the prefetching degree to maximize its coverage, as suggested. Additionally, we lowered the confidence threshold to 0.05 to better showcase Hyperion’s prefetching accuracy and coverage (see Section 4). Other parameters, such as *max prefetch count*, *max global counter value*, and *entry length*, are kept consistent with those used in the paper.

Table 5 and Table 6 present a comparison between Hyperion and the two built-in prefetchers, Stride Prefetcher and AMPM Prefetcher, across two benchmarks: gemm and shellsort. As the results demonstrate, Hyperion outperforms the Stride Prefetcher in both benchmarks and shows performance that is comparable to the AMPM Prefetcher.

In the gemm benchmark (Table 5), Hyperion achieves an IPC of 2.89, which is significantly higher than Stride’s 1.18 but slightly lower than AMPM’s 3.13. While Hyperion has lower accuracy and coverage than AMPM, its performance is still impressive, with a coverage rate of 95.17% and an accuracy of 23.41%, demonstrating its ability to effectively predict memory accesses. Furthermore, the late count for Hyperion (7,916,613) is considerably lower than AMPM’s (31,154,152), indicating a more delicate prefetchings.

In the shellsort benchmark (Table 6), Hyperion achieves the highest IPC of 0.87, outperforming both Stride (0.39) and AMPM (0.69). Hyperion also shows the highest coverage at 89.42%, highlighting its superior ability to reduce cache misses. However, its accuracy (21.07%) is slightly lower than AMPM’s 23.75%, but it still maintains a competitive performance. The late count for Hyperion is higher than AMPM’s (47,762,795 vs. 31,094,045), suggesting that while Hyperion might issue more prefetchings, some of them may still be late, which could be a potential area for further optimization.

Overall, different prefetchers could be particularly competitive under certain memory access patterns. Hyperion generally outperforms the Stride Prefetcher and has a comparable performance with the AMPM prefetcher.

gemm	Stride	AMPM	Hyperion
IPC	1.183529	3.125727	2.890424
Accuracy	0.000255	0.073765	0.234058
Coverage	0.000002	0.983178	0.951748
Late Count	9965	31154152	7916613

Table 5: Comparison of Metrics for Different Prefetchers on the gemm Benchmark

shellsort	Stride	AMPM	Hyperion
IPC	0.392248	0.687109	0.868869
Accuracy	0.300289	0.237503	0.210745
Coverage	0.074525	0.708856	0.894235
Late Count	3000534	31094045	47762795

Table 6: Comparison of Metrics for Different Prefetchers on the shellsort Benchmark

4 Paper Summary: Hyperion Prefetcher

The paper introduces Hyperion, a delta prefetcher designed for both page-level and PC-based prefetching, offering high L1D coverage and accuracy. In this section, we provide a comprehensive review of the background and design of Hyperion, including an overview of related previous studies and detailed implementations of Hyperion as presented in both the paper and the project’s source code. As previously mentioned, our customized version of Hyperion has been simplified to facilitate easier software simulation. We will also compare our implementation with the original version to highlight the differences and adaptations made for this purpose.

4.1 Background

Due to the speed mismatch between the processor and the main memory, the overhead of frequent memory accesses would become unacceptable without a cache hierarchy. Data prefetchers play a crucial role in the cache hierarchy. Most programs exhibit predictable patterns in memory access, such as spatial and temporal locality. This suggests that efforts can be made to predict future memory accesses based on past access patterns. High-performance prefetchers typically operate in this way: they predict memory addresses for future accesses by examining historical memory access information when specific events occur. By loading the data into the cache before the processor actually accesses it, these prefetchers minimize processor stalls caused by cache misses. Logically, prefetchers can be classified based on two features: the method of recording historical access information and the strategy for using this information to make predictions. These two aspects will be discussed separately in the following sections.

4.1.1 Historical Access Information

Different types of prefetchers record historical access information in various ways, depending on the amount of information they aim to record, as well as the granularity of the information. Some lighter-weight prefetchers tend to record coarse-grained information, such as using the offset of an access address within a memory page as an index in the historical access table. This allows accesses with the same offset on different pages to be merged and recorded, enabling the implementation of a prefetching mechanism with very limited CPU and cache resources. However, this coarse-grained approach has obvious drawbacks. The recorded information is relatively rough, which can lead to a higher number of incorrect predictions, thus lowering the prefetch accuracy.

To make prefetchers smarter and more adaptable in recognizing patterns, fine-grained information is necessary. Many high-performance prefetchers now attempt to record historical access information in sufficient detail within the available space, such as indexing the historical access table based on the memory access instruction address (i.e. PC) or the virtual page number (VPN). This approach allows for better differentiation of memory accesses and more precise and sensitive detection of various memory access patterns, ultimately improving prefetch accuracy. However, when the available storage space is fixed, fine-grained recording methods may lead to insufficient coverage. Therefore, this paper aims to enhance prefetcher performance by recording fine-grained access information across multiple dimensions, improving coverage while maintaining accuracy.

4.1.2 Prefetching Strategy

Currently, many prefetchers employ a mechanism called delta prefetching. During prefetching, the prefetchers dynamically calculate the stride (delta) patterns of memory accesses by querying the historical access table and select the most reliable stride for prefetching. Building on this, Hyperion further considers the fetch latency, incorporating time information from the historical access table to achieve timely delta prefetching. Since the project documentation provides a detailed explanation of the principles of both delta prefetching and timely delta prefetching, we will not go into further detail here. In the next section, we will thoroughly introduce the implementation and working process of Hyperion.

4.2 Design

As discussed in the previous section, Hyperion achieves timely delta prefetching by recording fine-grained historical access information, including the PC of the instruction, the VPN, the access address, and the time of access. The highlight lies in recording historical access information simultaneously in two dimensions: the access instruction's PC and the VPN. Through a sophisticated data structure design, Hyperion combines the information from both dimensions in one table, enabling the effective recording of fine-grained, multi-dimensional information within the constraints of storage space.

Through extensive experiments, the authors of the paper found that, although fine-grained information can effectively improve prefetch accuracy, it often results in local information (based on PC or VPN), which can make it difficult to detect and predict memory access patterns with larger strides, ultimately leading to reduced prefetch coverage. To address this, the paper proposes that recording fine-grained information from different dimensions, such as both PC-based and VPN-based accesses, can enhance the diversity of recorded memory access patterns in terms of access patterns. This approach improves coverage while maintaining accuracy. According to the paper, the specific design and implementation of Hyperion can be divided into four parts, which will be introduced separately below.

4.2.1 Recording Historical Access

Logically, historical access information is recorded separately in two tables: the PC History Table (PCHT), indexed by the program counter (PC) of the access instruction, and the Page History Table (PageHT), indexed by the VPN. Each entry in both PCHT and PageHT is organized as a first-in-first-out (FIFO) queue. In the PCHT, each element of the queue contains the memory access address and access time, while in the PageHT, each element contains the memory access address offset within the page and access time. Thus, both tables essentially record the address and time of memory accesses. Each queue has a maximum of 8 entries, and when the queue overflows, elements are evicted according to the FIFO principle. This means that each entry records the last 8 accesses corresponding to that specific PC or VPN. In this project, Hyperion is implemented in accordance with the description above. However, due to the need to consider cache size limitations in hardware, the paper proposes merging the two tables into a single table, called the Global History Buffer (GHB), to reduce redundant information and save cache space. Additionally, a cyclic FIFO queue is established in the GHB using the PC index

table (PCiT) and the Page index table (VPNiT). And these three tables are integrated into a single structure called the Double-indexed Global History Buffer (DiGHB).

4.2.2 Recording Deltas

The approach for recording deltas is essentially similar to that of recording historical access information. A simple implementation would store the calculated deltas in two separate tables: one based on the PC (PCDT) and the other based on the VPN (PageDT). Each entry in these tables contains a global counter and an array composed of delta values and local counter pairs. Similar to the history tables (HT), the two delta tables (DT) are also organized together in the paper, referred to as the Unified Delta Table (U.DT).

4.2.3 Learning Timely Deltas

Before learning the timely delta, it is necessary to first determine the latency corresponding to a cache miss or prefetch hit. In general, the latency for a cache miss is the time from the cache miss to the demand hit, while the latency for a prefetch hit is the time from the prefetch issue to the prefetch hit. Once the latency is determined, the best request time can be calculated. The prefetcher then queries the HT to find the first memory access before the best request time and calculates the delta (i.e., the appropriate prefetch stride).

After obtaining the delta, the corresponding delta array entry is updated (either adding the newly learned delta or updating the local counter for that delta), along with the global and local counters. If the global counter exceeds the preset threshold (16), it is halved along with all the corresponding local counters. If the number of array elements exceeds 8, the delta with the smallest local counter is evicted. This logic in the paper aligns with our implementation.

4.2.4 Issuing Strategy

When a cache hit occurs (including both demand hits and prefetch hits) or a demand miss happens, the prefetcher will perform prefetching. The prefetch stride comes from the Delta Table (DT), where the prefetcher locates the corresponding entry based on the PC and VPN, and selects a delta that meets certain conditions to be used as the stride for prefetching. In the paper, the condition for selecting a delta is that the global counter for the entry is greater than half of the preset maximum value (16), and the confidence in the delta is determined by dividing each delta's local counter by the global counter. Based on this confidence, along with the real-time L1D prefetch accuracy, the prefetcher decides whether to issue the prefetch and whether to load the data into the L1D cache or the L2 cache, thereby implementing dynamic cache-level selection.

In our implementation, we made some simplifications and adjustments based on the documentation's suggestions for this part: we did not implement dynamic cache-level selection. Instead, we only use the confidence to decide whether to prefetch data into the L1D cache, and we lowered the threshold for prefetching data into the L1D cache. The paper suggests a threshold of 0.8 to avoid reducing accuracy, but we significantly reduced it to 0.05 in order to increase the number of prefetches as much as possible, thus better demonstrating the performance of Hyperion. Additionally, we did not follow the suggestion in the documentation to increase the degree of prefetching, but instead chose to remain consistent with the paper to more accurately reflect Hyperion's prefetch prediction accuracy and coverage.